

COGNOME:

NOME:

MATRICOLA:

Appello di Algoritmi e Strutture Dati (9 CFU), 5 Giugno 2026

Parte I - Usare voto prova parziale - SI() NO()

Esercizio 1 [3 punti]Quale delle seguenti sequenze di funzioni è tale che ogni funzione è O della successiva?

(A) $\frac{n}{\log n}, n^{0.999}, n \log n, \binom{n}{2}, \binom{n}{n-3}$

(B) $\sum_{i=0}^{\log n} 2^i, \frac{n}{\log n}, 2^{\sqrt{n}}, \sum_{i=0}^n 2^i, 2^n$

~~(C)~~ $n^{\log_{2026} 2025}, \sum_{i=1}^n \log i, n \log n, 2^{(\log n)^{2026}}, n^{\sqrt{n}}$

(D) Nessuna delle altre

Esercizio 2 [4 punti]Si consideri il seguente algoritmo, in cui l'input è un intero positivo n . Qual è il numero di chiamate alla funzione f ?**Algorithm** ALGO(n)1: **if** $n = 1$ **then**2: $f()$ 3: **end if**4: **if** $n > 1$ **then**5: ALGO($n/2$)6: ALGO($n/2$) $\left. \begin{array}{l} 2\tau(n/2) \end{array} \right\}$ 7: $i = 1$ 8: **while** $i \leq n$ **do** $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow \dots \rightarrow n$ 9: $j = 1$ 10: **while** $j \leq \log i$ **do**11: $f()$ 12: $j = j + 1$ 13: **end while**14: $i = 2i$ 15: **end while**16: **end if**

$i=1 \Rightarrow j = \log 1 = 0$

$i=2 \Rightarrow j = \log 2 = 1$

$i=4 \Rightarrow j = \log 4 = 2$

$i=n \Rightarrow j = \log n$

$0 + 1 + 2 + \dots + \log n = \frac{\log n (\log n + 1)}{2} = \frac{\log^2 n + \log n}{2} \approx \Theta(\log^2 n)$

(A) $\Theta(\log n)$ ~~(B)~~ $\Theta(n)$ (C) $\Theta(n \log n)$

(D) Nessuna delle altre

$\tau(n) = 2\tau(n/2) + \Theta(\log^2 n)$

$a=2 \quad b=2 \quad n^{\log_2 2} = n > \log^2 n$

$\tau(n) = \Theta(n)$

Esercizio 3 [3 punti]

Quale delle seguenti affermazioni è vera?

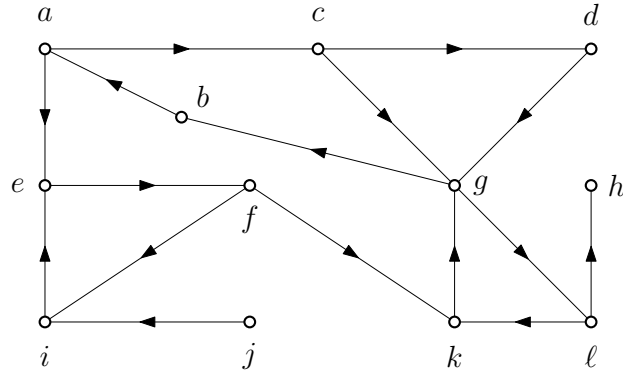
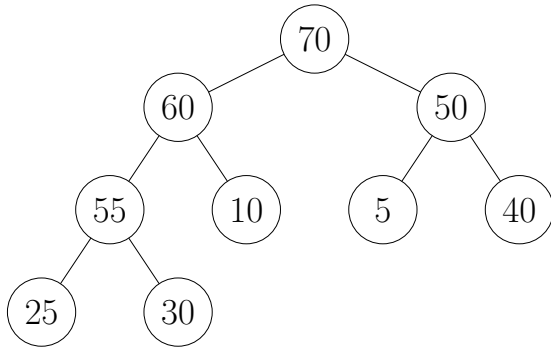
(A) Per trovare massimo e minimo di un array di n interi è necessario effettuare almeno $2(n-1)$ confronti.

(B) L'albero di decisione per l'ordinamento di 5 interi corrispondente all'algoritmo MERGE-SORT ha 60 foglie.

(C) Vogliamo ordinare array formati da n interi positivi, tutti minori di $2^{(\log n)^3}$. Allora, per input di questo tipo, il tempo di esecuzione nel caso peggiore di RADIX-SORT è asintoticamente inferiore (o) rispetto a quello di MERGE-SORT.~~(D)~~ Nessuna delle altre.

Esercizio 4 [3 punti]

Si consideri l'albero in figura, che rappresenta un max-heap A . La rappresentazione di A tramite array dopo chiamate, nell'ordine, a $\text{MAX-HEAP-INSERT}(A, 20)$, $\text{HEAP-EXTRACT-MAX}(A)$, $\text{MAX-HEAP-INSERT}(A, 65)$, e $\text{HEAP-EXTRACT-MAX}(A)$ è: ~~(A)~~ $\langle 60, 55, 50, 30, 20, 5, 40, 25, 10 \rangle$ (B) $\langle 60, 20, 50, 30, 55, 5, 40, 25, 10 \rangle$ (C) $\langle 60, 55, 50, 30, 20, 40, 5, 25, 10 \rangle$ (D) Nessuna delle altre



Esercizio 5 [4 punti]

Si consideri il grafo diretto G in figura. Si effettui una visita in profondità di G a partire da a in cui i vertici sono ordinati alfabeticamente, così come ciascuna lista di adiacenza. Quale delle seguenti affermazioni è vera?

- ~~(A)~~ Nessuna delle altre.
 (B) (j, i) è un arco in avanti.
 (C) G ha 4 componenti fortemente connesse.
 (D) L'insieme degli archi all'indietro è $\{(i, e), (k, g), (c, g), (b, a)\}$.

Esercizio 6 [3 punti]

Quale delle seguenti affermazioni è vera?

- (A) Nessuna delle altre.
 (B) Costruiamo un albero binario di ricerca inserendo, nell'ordine, le chiavi 1, 4, 3, 8, 2, 9, 5, 7, e poi rimuovendo la chiave 4. Allora il figlio destro di 5 è 7.
 (C) Se un grafo pesato G ammette un unico albero di connessione minimo, allora non esistono due archi di G aventi lo stesso peso.
~~(D)~~ Dato un grafo diretto $G = (V, E)$, il grafo diretto $G' = (V', E')$ è tale che $V' = V$ e, per ogni $u, w \in V'$, si ha che $(u, w) \in E'$ se e solo se esiste $v \in V$ tale che $\{(u, v), (v, w)\} \subseteq E$. Se A è la matrice di adiacenza di G , allora la matrice di adiacenza di G' è la matrice booleana ottenuta da A^2 sostituendo ogni elemento positivo con 1.

SBARRAMENTO - 12 punti sugli esercizi precedenti

Esercizio 7 [8 punti] Si consideri l'algoritmo di Dijkstra. Se ne descriva brevemente l'idea, e si forniscano pseudo codice e tempo di esecuzione nel caso peggiore.

Esercizio 8 [4 punti] La rete autostradale del paese P è rappresentata da un grafo G i cui vertici sono le n città principali e i cui archi sono gli m tratti di autostrada tra coppie di città (G non contiene archi paralleli). Per scoraggiare l'utilizzo dell'auto¹, il governo di P decide di rendere ciascuno di questi m tratti a senso unico, i.e., orienta gli archi di G , ottenendo così un grafo diretto G' . Il governo asserisce che, data una qualsiasi coppia di città $u, v \in V(G')$, è comunque possibile raggiungere u da v o v da u in G' . Fornire un algoritmo che verifichi questa asserzione in $O(n + m)$ sapendo che G' è aciclico. Dimostrare poi la correttezza dell'algoritmo.

¹A torto o a ragione, l'ardua sentenza al lettore.

Quale delle seguenti sequenze di funzioni è tale che ogni funzione è O della successiva?

(A) $\frac{n}{\log n}$, $n^{0.999}$, $n \log n$, $\binom{n}{2}$, $\binom{n}{n-3}$

(B) $\sum_{i=0}^{\log n} 2^i$, $\frac{n}{\log n}$, $2^{\sqrt{n}}$, $\sum_{i=0}^n 2^i$, 2^n

(C) $n^{\log_{2026} 2025}$, $\sum_{i=1}^n \log i$, $n \log n$, $2^{(\log n)^{2026}}$, $n^{\sqrt{n}}$

(D) Nessuna delle altre

A) $\frac{n}{\log n} > n^{0.999}$

B) $\sum_{i=0}^{\log n} 2^i > \frac{n}{\log n}$

$\sum_{i=0}^{\log n} 2^i \approx 2^{\log n} = n$

C) $n^{\log_{2026} 2025} \leq \sum_{i=1}^n \log i \leq n \log n \leq 2^{(\log n)^{2026}} \leq n^{\sqrt{n}}$

$n^{\log_{2026} 2025} \xrightarrow[n]{\text{Costante}}$

$\sum_{i=1}^n \log i = \log(n!) \approx n \log n$

$2^{(\log n)^{2026}} = 2^{\log n \cdot (\log n)^{2025}} = n^{(\log n)^{2025}}$

$n^{\sqrt{n}}$ Confronto $(\log n)^{2025}$, $\sqrt{n}$

$\log(\log n)^{2025}$ $\log \sqrt{n}$

$2025 \log \log n$ $\log n^{\frac{1}{2}}$

$2025 \log \log n$ $\frac{1}{2} \log n$

$\log \log n < \log n$

Esercizio 7 [8 punti] Si consideri l'algoritmo di Dijkstra. Se ne descriva brevemente l'idea, e si forniscano pseudo codice e tempo di esecuzione nel caso peggiore.

Dijkstra: È un algoritmo greedy utilizzato per risolvere il problema dei Cammini minimi da sorgente singola su un grafo pesato e orientato (o non orientato), con peso Positivo (>0)

Il Funzionamento:

1. Viene mantenuta una Coda di min-priorità Q contenente tutti i vertici del grafo, dove la chiave di ciascun vertice è la sua stima corrente del cammino minimo $c.v.d$
2. Ad ogni passo, l'algoritmo estrae da Q il vertice u con la stima di distanza minima assoluta (greedy) e lo inserisce nell'insieme dei nodi
3. L'algoritmo esegue il rilassamento di tutti gli archi uscenti da u , Per ogni vicino v , controlla se il cammino passante per u è più conveniente della stima attuale $v.d$. Se lo è, aggiorna $v.d$ e il predecessore $v.p$

DIJKSTRAC(G, s)

INITIALIZE-SINGLE-SOURCE(G, s) $\rightarrow O(|V|)$

$S = \emptyset$

$Q = G.V$

Mentre $Q \neq \emptyset$ do

$u = \text{EXTRACT-MIN}(Q)$ $\rightarrow O(|V| \log |V|)$

$S = S \cup \{u\}$

Per ogni vertice $v \in G.Adj[u]$ do

RELAX(u, v, w)

DECREASE-KEY($Q, v, v.d$) $\rightarrow O(|E| \log |V|)$, $|E| \geq |V| - 1 \rightarrow$ Graf connesso

$\text{tot} = O(|E| \log |V| + |V| \log |V|) = O(|E| \log |V|)$

Esercizio 8 [4 punti] La rete autostradale del paese P è rappresentata da un grafo G i cui vertici sono le n città principali e i cui archi sono gli m tratti di autostrada tra coppie di città (G non contiene archi paralleli). Per scoraggiare l'utilizzo dell'auto¹, il governo di P decide di rendere ciascuno di questi m tratti a senso unico, i.e., orienta gli archi di G , ottenendo così un grafo diretto G' . Il governo asserisce che, data una qualsiasi coppia di città $u, v \in V(G')$, è comunque possibile raggiungere u da v o v da u in G' . Fornire un algoritmo che verifichi questa asserzione in $O(n+m)$ sapendo che G' è aciclico. Dimostrare poi la correttezza dell'algoritmo.

¹A torto o a ragione, l'ardua sentenza al lettore.

Eseguiamo un ordinamento topologico sul DAG G' e otteniamo una sequenza ordinata di vertici.

Effettuiamo una scansione lineare della sequenza. Per ogni posizione i da 1 a $n-1$, controlliamo se nel grafo originale esiste l'arco diretto che va dal nodo corrente al successivo immediato.

se per ogni i l'arco (v_i, v_{i+1}) è presente nel grafo, l'asserzione del governo è vera.

se anche solo per un indice i l'arco (v_i, v_{i+1}) non esiste, l'asserzione è falsa.

Dimostrazione:

⇒: se $\forall$ coppia consecutiva esiste l'arco (v_i, v_{i+1}) , allora abbiamo un cammino completo che tocca tutti i nodi.

⇐: supponiamo che l'algoritmo fallisce perché per un determinato indice i manca l'arco (v_i, v_{i+1}) ,
per assurdo
quando il grafo è ordinato topologicamente, non può esistere un cammino che va da v_{i+1} a v_i ,
se manca l'arco (v_i, v_{i+1}) non può esistere alcun cammino tra i 2 nodi che passi per nodi intermedi,
Poiché qualsiasi percorso dovrebbe violare le regole di precedenza del DAG.
Di conseguenza, i nodi v_i e v_{i+1} risultano non raggiungibili in alcuna direzione.